

HW02 Robotic Technology

Narjes ghazanfari.401109177

December 13, 2025

1 Theoretical

1.1 Driving Motion Model Equations

this is our general shape of equation.

$$\begin{bmatrix} x_{1,t} \\ x_{2,t} \\ x_{3,t} \end{bmatrix} = g(x_{t-1}, u_t, \varepsilon_t) = \begin{bmatrix} x_{1,t-1} + u_{1,t} \cos x_{3,t-1} dt \\ x_{2,t-1} + u_{1,t} \sin x_{3,t-1} dt \\ x_{3,t-1} + u_{2,t} dt \end{bmatrix} + \varepsilon_t \quad (1)$$

and based on it over equation will be:

$$\begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} x_{t-1} + V_{x_t} \cos \theta_{t-1} dt \\ y_{t-1} + V_{x_t} \sin \theta_{t-1} dt \\ \theta_{t-1} + \omega_{z_t} dt \end{bmatrix} + \varepsilon_t \quad (2)$$

and based on the given data ω_R and ω_L will be:

$$\omega_R = \frac{1}{r} \left(V_x + \frac{w}{2} \omega_z \right) \frac{60}{2\pi} \quad (3)$$

$$\omega_L = \frac{1}{r} \left(V_x - \frac{w}{2} \omega_z \right) \frac{60}{2\pi} \quad (4)$$

and based on them V_x and ω_z equal with:

$$V_x = \frac{r\pi}{60} (\omega_R + \omega_L) \quad (5)$$

$$\omega_z = \frac{2\pi}{60} \frac{r}{w} (\omega_R - \omega_L) \quad (6)$$

1.2 Driving measurement Model Equations

In this part we calculate the y_t :

$$y_t = \begin{bmatrix} x_t^{vo} \\ y_t^{vo} \\ \theta_t^{imu} \end{bmatrix} \quad (7)$$

or we can show it with this way:

$$y_t = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \delta_t \quad (8)$$

1.3 Deriving Extended Kalman Filter

State and inputs

$$x_t \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}, \quad u_t \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} \quad (9)$$

Motion model (nonlinear)

$$x_t = g(x_{t-1}, u_t) + \epsilon_t, \quad (10)$$

$$\epsilon_t \sim \mathcal{N}(0, R_t) \quad (11)$$

$$g(x_{t-1}, u_t) = \begin{bmatrix} x_{t-1} + v_t \cos(\theta_{t-1}) \Delta t \\ y_{t-1} + v_t \sin(\theta_{t-1}) \Delta t \\ \theta_{t-1} + \omega_t \Delta t \end{bmatrix} \quad (12)$$

Measurement model (linearization)

$$y_t = h(x_t) + \delta_t, \quad \delta_t \sim \mathcal{N}(0, Q_t) \quad (13)$$

$$h(x_t) \approx h(\bar{\mu}_t) + \left. \frac{\partial}{\partial x_t} h(x_t) \right|_{x_t = \bar{\mu}_t} (x_t - \bar{\mu}_t) \quad (14)$$

$$= h(\bar{\mu}_t) + H_t \cdot (x_t - \bar{\mu}_t) \quad (15)$$

EKF Prediction update

$$G_t = \left. \frac{\partial}{\partial x_{t-1}} g(x_{t-1}, u_t) \right|_{x_{t-1} = \mu_{t-1}} \quad (16)$$

$$\bar{\mu}_t = g(\mu_{t-1}, u_t) \quad (17)$$

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t \quad (18)$$

EKF Measurement update

$$H_t = \left. \frac{\partial}{\partial x_t} h(x_t) \right|_{x_t = \bar{\mu}_t} = I_3 \quad (19)$$

Predicted mean

$$\bar{\mu}_t = g(\mu_{t-1}, u_t) = \begin{bmatrix} \mu_{x,t-1} + v_t \cos(\mu_{\theta,t-1}) \Delta t \\ \mu_{y,t-1} + v_t \sin(\mu_{\theta,t-1}) \Delta t \\ \mu_{\theta,t-1} + \omega_t \Delta t \end{bmatrix} \quad (20)$$

Jacobian of motion model wrt state (a.k.a. G_t)

$$G_t \left. \frac{\partial g(x, u_t)}{\partial x} \right|_{x = \mu_{t-1}} = \begin{bmatrix} 1 & 0 & -v_t \sin(\mu_{\theta,t-1}) \Delta t \\ 0 & 1 & v_t \cos(\mu_{\theta,t-1}) \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (21)$$

2 Implementation

2.1 URDF Update

The robot's URDF is extended by introducing two independent RPM-driven motors. This modification enables the robot to receive and execute distinct RPM commands for the left and right wheels, establishing the foundation for motion control.

```
<!-- Low Level Control -->
<gazebo>
  <plugin filename="gz-sim-joint-controller-system"
    name="gz::sim::systems::JointController">
    <joint_name>drivewhl_l_joint</joint_name>
    <initial_velocity>0.0</initial_velocity>
    <topic>left_motor_rpm</topic>
  </plugin>
</gazebo>
<gazebo>
  <plugin filename="gz-sim-joint-controller-system"
    name="gz::sim::systems::JointController">
    <joint_name>drivewhl_r_joint</joint_name>
    <initial_velocity>0.0</initial_velocity>
    <topic>right_motor_rpm</topic>
  </plugin>
```

Figure 1: URDF Update

2.2 IMU Noise

A realistic noise model is incorporated into the IMU specification within the URDF. This addition ensures that sensor readings reflect practical measurement uncertainties, thereby improving the fidelity of subsequent state-estimation processes.

```
<!-- acceleration noise -->
<accel>
  <noise type="gaussian">
    <mean>0.0</mean>
    <stddev>0.02</stddev>
    <bias_mean>0.0</bias_mean>
    <bias_stddev>0.01</bias_stddev>
  </noise>
</accel>

<!-- angular velocity noise -->
<gyro>
  <noise type="gaussian">
    <mean>0.0</mean>
    <stddev>0.01</stddev>
    <bias_mean>0.0</bias_mean>
    <bias_stddev>0.01</bias_stddev>
  </noise>
</gyro>
</sensor>
</gazebo>
```

Figure 2: IMU Noise

2.3 Controller Node

A C++ controller node is implemented to process incoming `cmd_vel` messages. The node converts linear and angular velocity commands into corresponding left and right motor RPM values, enabling direct actuation of the robot's drivetrain.

2.4 Prediction Node

A Python prediction node is developed to perform the motion-model update of the filter. It utilizes essential robot parameters—such as wheel separation, wheel radius, and the command topic—to compute the predicted robot state at each time step.

```
header:
  stamp:
    sec: 8
    nanosec: 256000000
  frame_id: odom
child_frame_id: base_link
pose:
  pose:
    position:
      x: 3.259204149246216
      y: 2.174001455307007
      z: -0.83902508020401
    orientation:
      x: 0.0003343423695772527
      y: -0.0004726149541642614
      z: 0.39586609476681955
      w: 0.9183080637043064
  covariance:
    - 0.0
    - 0.0
    - 0.0
    - 0.0
```

Figure 3: Prediction Data Sample

2.5 Measurement Node

The measurement node subscribes to IMU and visual-odometry inputs and fuses them using the provided algorithm. It computes the combined measurement model and publishes the resulting fused observations for use in the estimation pipeline.

```
- 0.0
- 0.0
- 0.0
header:
  stamp:
    sec: 1765544089
    nanosec: 926387934
  frame_id: odom
child_frame_id: base_link
pose:
  pose:
    position:
      x: 0.0
      y: 0.0
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.4145970877015424
      w: 0.910005085078869
  covariance:
    - 0.0
    - 0.0
    - 0.0
    - 0.0
```

Figure 4: Measurement Data Sample

2.6 EKF Node

An Extended Kalman Filter (EKF) node is implemented to integrate both prediction and measurement updates. The node produces a refined estimate of the robot state and publishes it in the standard Odometry message format for downstream modules and its data sample is like the prediction and measurement parts.

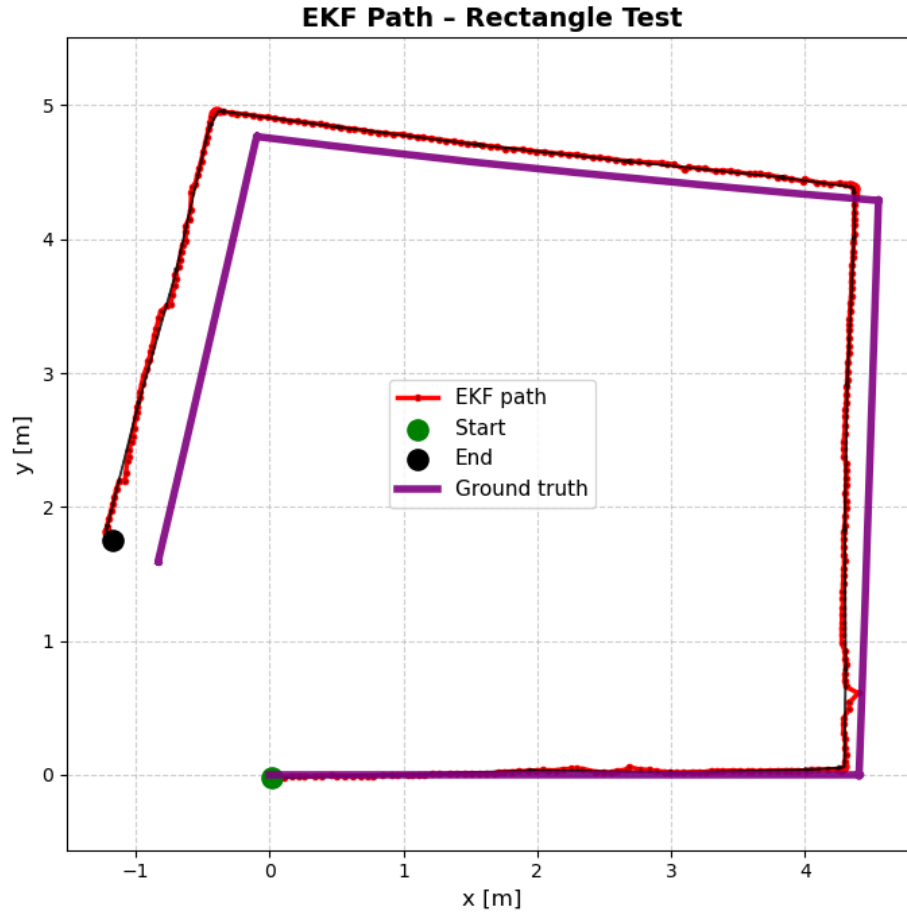


Figure 5: EKF and Ground Path Diagram

2.7 Test Node

A dedicated test node commands the robot to track a predefined rectangular path using the EKF-derived odometry. During execution, it records and visualizes the real trajectory, measurement-based path, motion-model output, and EKF-estimated path in RViz through `Path` messages.

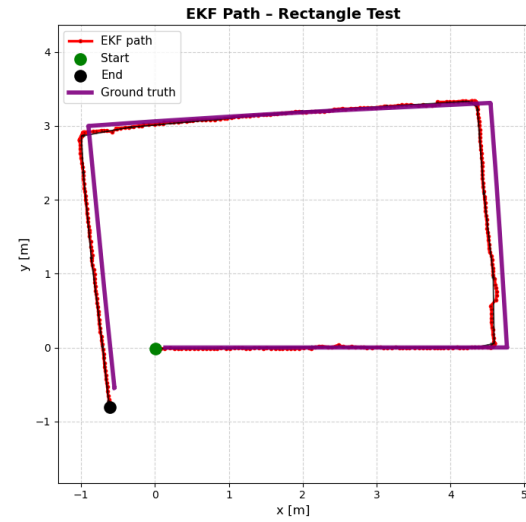
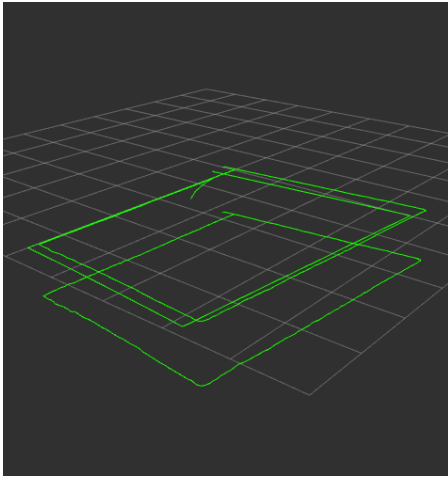
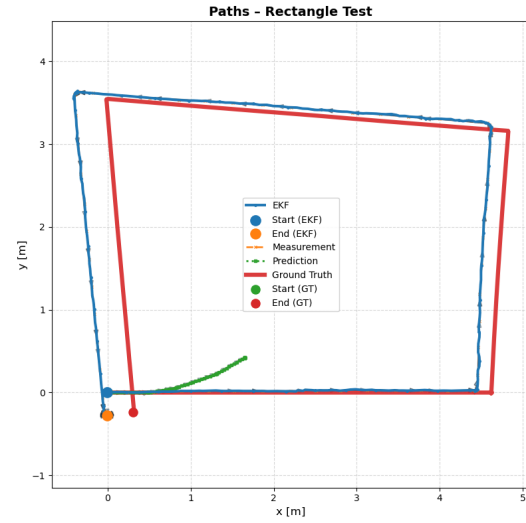


Figure 6: Test Node Results

2.8 Rosbag Analysis

Rosbag is employed to record the experiment data, which is subsequently analyzed using Foxglove or Lichtblick. These tools facilitate replay, inspection, and detailed visualization of the trajectories and state-estimation performance. In here after saving data by Rosbag we will open them by Foxglove and the results located in the continue.

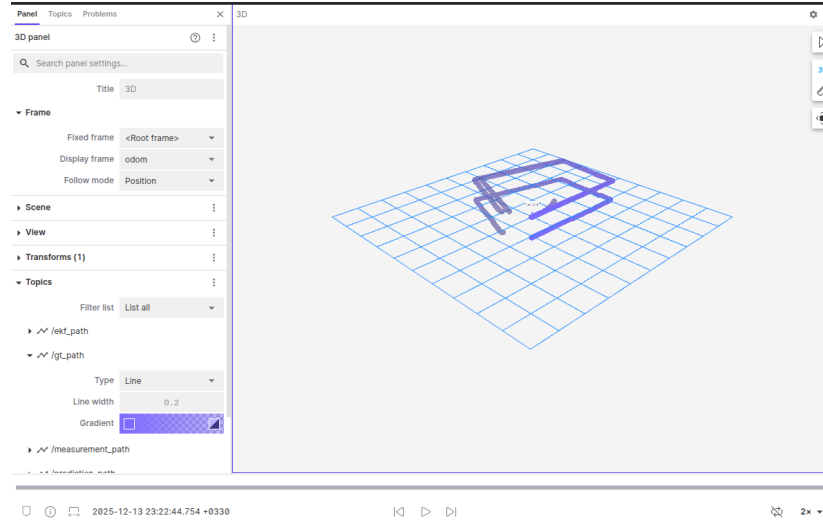


Figure 7: Analyzing Paths by Using Foxglove

Code Availability

The source code associated with this work is publicly available at: https://github.com/Narjes-Gh2024/HW02_Robotic_Technology.git